WEB PLATFORM

HTTP Caching Visual Guide

Cache-Control, validators, and CDN interplay — serve bytes once, reuse them a million times.

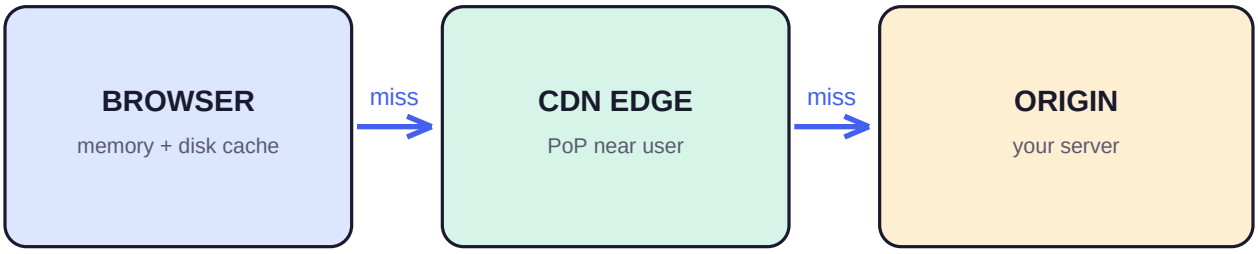
ANIMATED WALKTHROUGH

Step through pages to play the animation

STEP 1/6

The Cache Hierarchy

Every layer that can answer without bothering origin saves time and money.



Hits get faster and cheaper as you move left. Design for edge hits.

Shared caches (CDN) vs private caches (browser): directives decide who may store what.

STEP 2/6

Cache-Control Directives

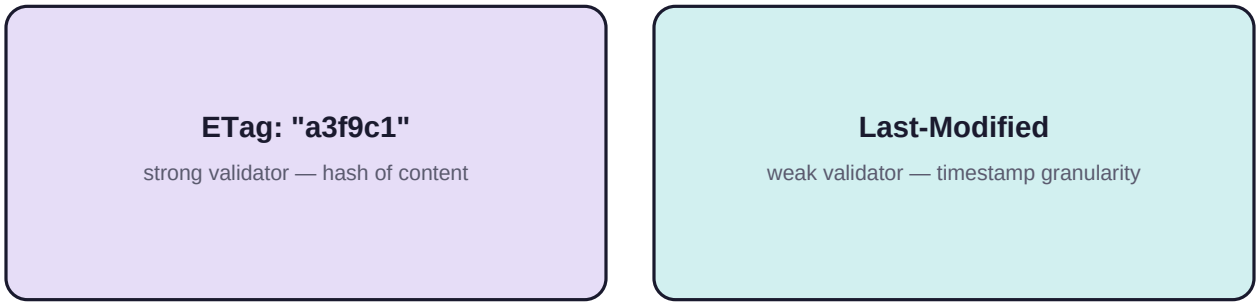
The response header that is really a policy language.

max-age=31536000	fresh for a year — immutable assets
no-cache	store it, but revalidate every time
no-store	never store — secrets, sensitive data
public / private	CDN may cache / browser-only
stale-while-revalidate	serve stale instantly, refresh in background

STEP 3/6

Validators: ETag & Last-Modified

Fingerprints that let caches ask 'did this change?' instead of re-downloading.



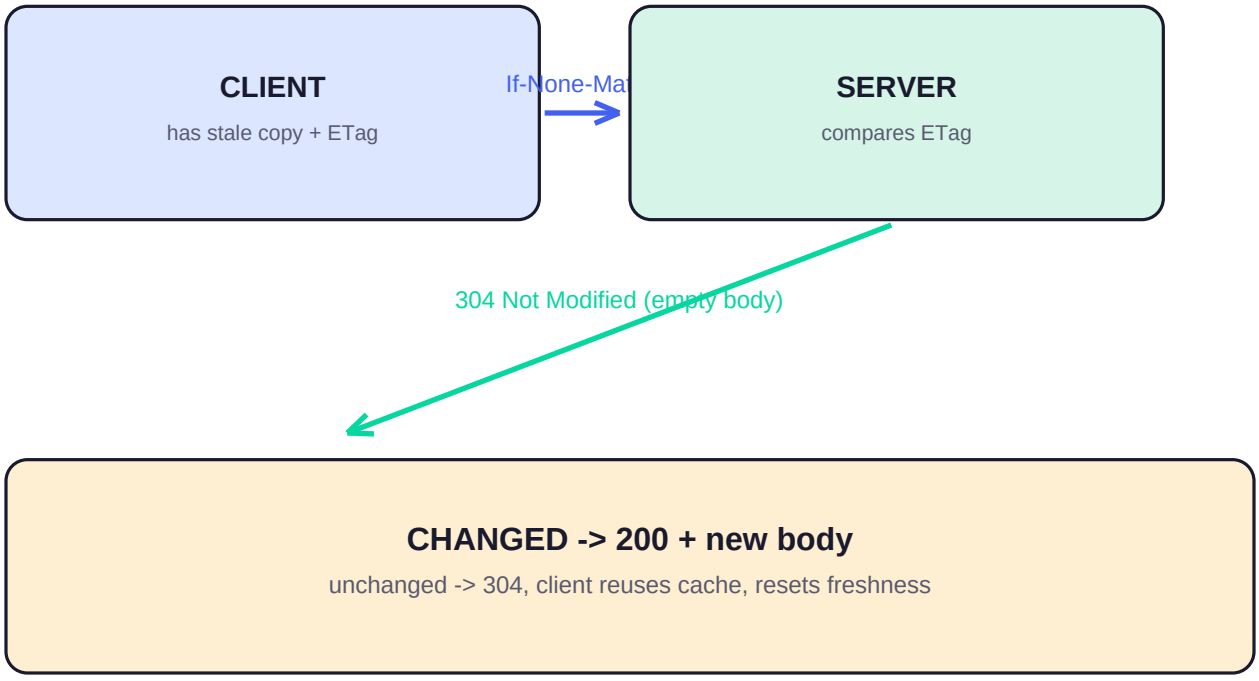
Strong validators compare bytes; weak ones (W/) compare 'semantically same'.

Content-hashed filenames (app.a3f9c1.js) are the ultimate validator: the name IS the version.

STEP 4/6

Conditional Requests: The 304 Dance

Revalidate cheaply — a 304 means 'your copy is still good'.

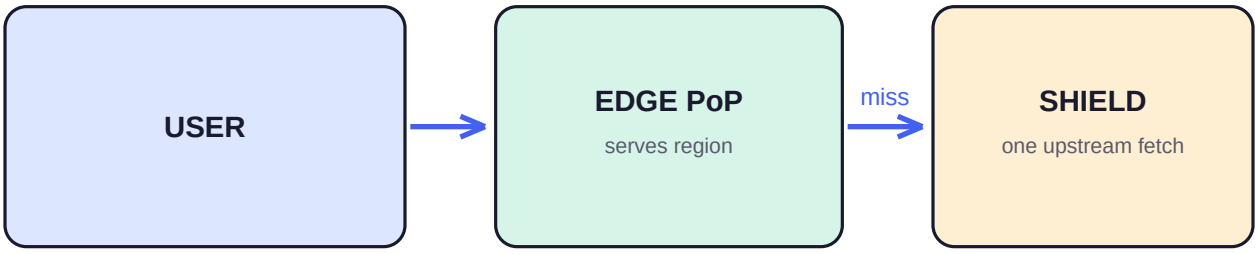


A 304 saves the body bytes but still costs a round trip — pair with long max-age for hot assets.

STEP 5/6

CDN Interplay

Edge caches, shielding, and tiered caching multiply hit rates.



Without shielding, 50 edge misses = 50 origin hits (thundering herd).

STEP 6/6

Invalidation Strategies

The two hard problems: naming things, cache invalidation, and off-by-one errors.

Content hashing	new content -> new filename; old URLs stay cached forever
Short TTL + SWR	HTML shell: max-age=60, stale-while-revalidate
Purge API	explicit CDN purge on deploy — use sparingly
Versioned paths	/v2/... prefixes — atomic cutover

STEP ✓/✓

Key Takeaways

HTTP Caching — Remember

- Caches layer: browser, CDN edge, shield, origin — design for hits at the edge.
- Cache-Control is a policy language: max-age, no-cache, no-store, public/private, SWR.
- ETag and Last-Modified let caches revalidate with cheap conditional requests.
- 304 Not Modified reuses the cached copy; content-hashed filenames beat validators entirely.
- Shielding collapses edge misses into one origin fetch; Vary splits cache variants.
- Invalidate by renaming (hashing), short TTLs on HTML, and purge APIs as a last resort.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev